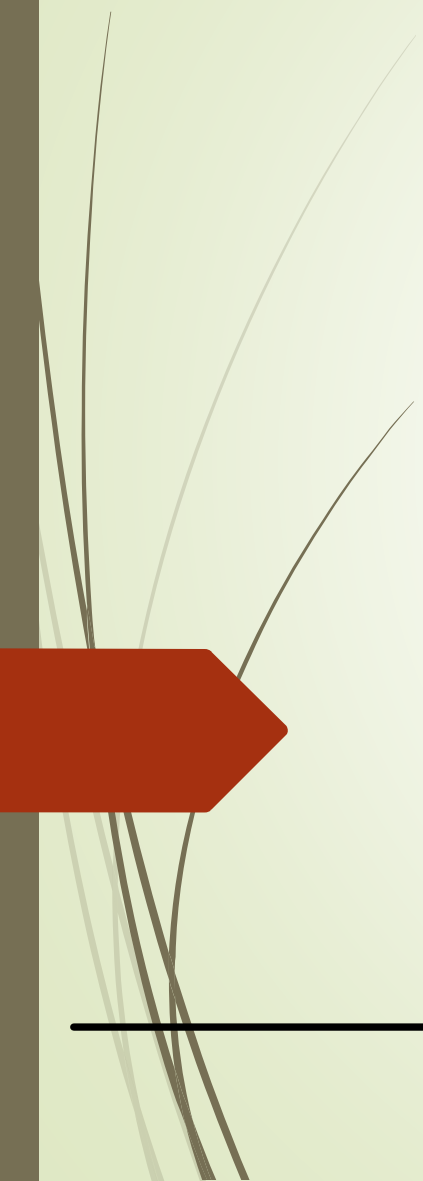

Nhắc lại kiến thức bài trước

Lập trình căn bản



Đối tượng

Đối tượng là gì?

"Tất cả mọi thứ đều là đối tượng"

-Alan Kay





Triết lý căn bản của ngôn ngữ Python được trình bày trong tài liệu The Zen of Python (PEP 20), có dạng thơ Haiku, tóm gọn như sau:

- Beautiful is better than ugly.
- Explicit is better than implicit.
- Simple is better than complex.
- Complex is better than complicated.
- Readability counts.

- Đẹp dễ tốt hơn xấu xí
- Minh bạch tốt hơn ngầm định
- Đơn giản tốt hơn phức tạp
- Phức tạp tốt hơn rắc rối
- Tính dễ đọc rất quan trọng.

✗ NGÔN NGỮ KHÁC (C/JAVA)



C

```
int temp = a;  
a = b;  
b = temp;
```

VS

✓ PYTHON (PYTHONIC)



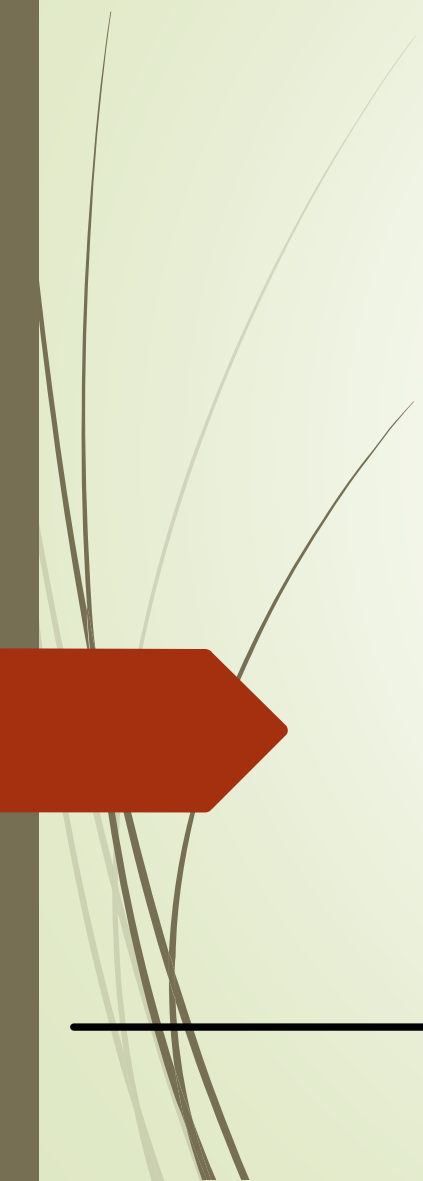
Python

```
a, b = b, a # 1 dòng!
```

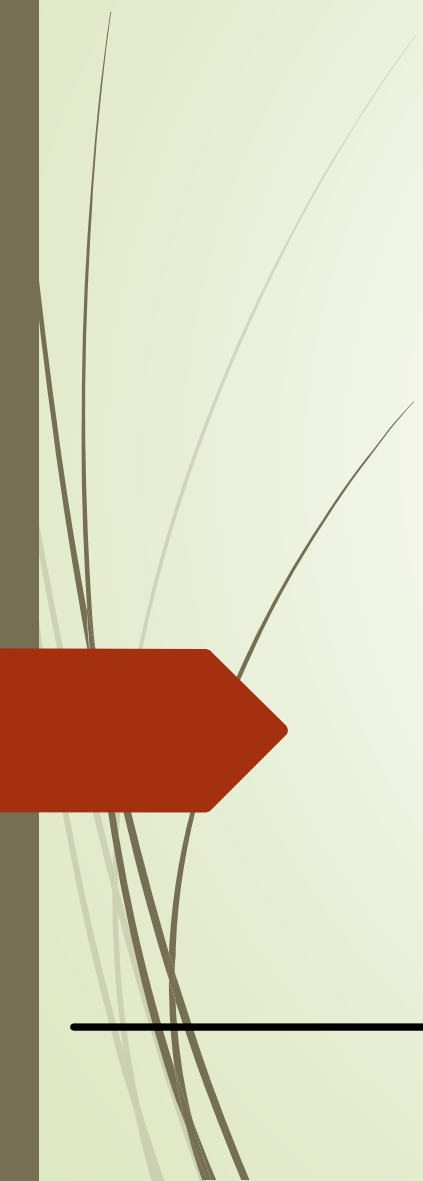
Quy ước đặt tên

- Quy ước với định danh (naming convention ³):
 - Bắt đầu bằng chữ cái
 - Gói (package): tất cả sử dụng chữ thường
`theexample`
 - Lớp (Class): viết hoa chữ cái đầu tiên trong các từ ghép lại
`TheExample` - PascalCase
 - Phương thức/thuộc tính (method/attribute): Bắt đầu bằng chữ thường, viết hoa chữ cái đầu tiên trong các từ còn lại
`theExample` - camelCase
 - Hằng (constants): Tất cả viết hoa
`THE_EXAMPLE`

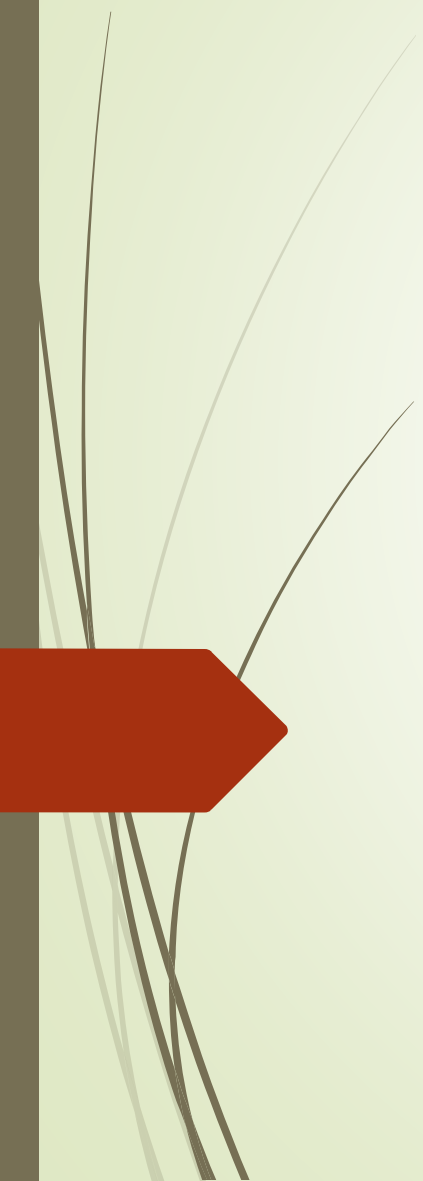
OOP03: Xây dựng lớp



Nội dung

1. Trừu tượng hóa dữ liệu
 2. Lớp và thành phần của lớp
 3. Xây dựng lớp
 4. Tạo và sử dụng đối tượng
 5. Thành viên hằng & tĩnh
 6. Biểu đồ lớp
- 
-

1



Trừu tượng hóa dữ liệu

Data abstraction

Trừu tượng hóa

- Abstraction
 - "a concept or idea not associated with any specific instance"*
 - "một khái niệm hoặc ý tưởng không liên quan đến bất kỳ trường hợp cụ thể"
- Giảm thiểu và tinh lọc các chi tiết nhằm tập trung vào một số khái niệm/vấn đề quan tâm tại một thời điểm.
 - Ví dụ: Các định nghĩa toán học: Ký hiệu **x** được dùng để thể hiện cho các phép nhân

Trừu tượng hóa

Quan sát thực tế

Đối tượng thực có nhiều chi tiết phức tạp



Lọc thông tin

Bỏ qua chi tiết không cần thiết



Trích xuất đặc tính

Chỉ giữ lại thông tin quan trọng



Xây dựng Lớp

Định nghĩa kiểu dữ liệu mới

Trừu tượng hóa điều khiển

Sử dụng các chương trình con (subprogram) và các luồng điều khiển (control flow) để ẩn đi độ phức tạp tính toán.

Ví dụ: $a := (1 + 2) * 5$ — lập trình viên không cần quan tâm đến thanh ghi hay bước tính nhị phân.

Trừu tượng hóa dữ liệu

Xử lý dữ liệu ở mức độ trừu tượng — tách biệt thuộc tính trừu tượng khỏi chi tiết thực thi.

Ví dụ: Kiểu dữ liệu int — ta dùng mà không cần biết cách lưu trữ bit bên trong.

Trừu tượng hóa dữ liệu

- Đối tượng trong thực tế phức tạp: điện thoại Nokia



- Cần đơn giản hóa, bỏ qua những chi tiết không cần thiết
- Chỉ “trích rút” lấy những thông tin liên quan, thông tin quan tâm, quan trọng với bài toán

Ví dụ: Điện thoại Nokia

- Những thông tin gì có thể cảm nhận được khi nhìn các "đối tượng" này?
 - Tất cả là điện thoại Nokia
 - Các điện thoại này có loại nắp trượt, có loại nắp gập, có loại dạng bar
 - Một số điện thoại là dòng doanh nhân, một số dòng âm nhạc, 3G...
 - Bàn phím loại tiêu chuẩn, QWERTY hoặc không có bàn phím
 - Màu sắc, chất liệu, kích cỡ... khác nhau
 - v.v...

Ví dụ: Điện thoại Nokia

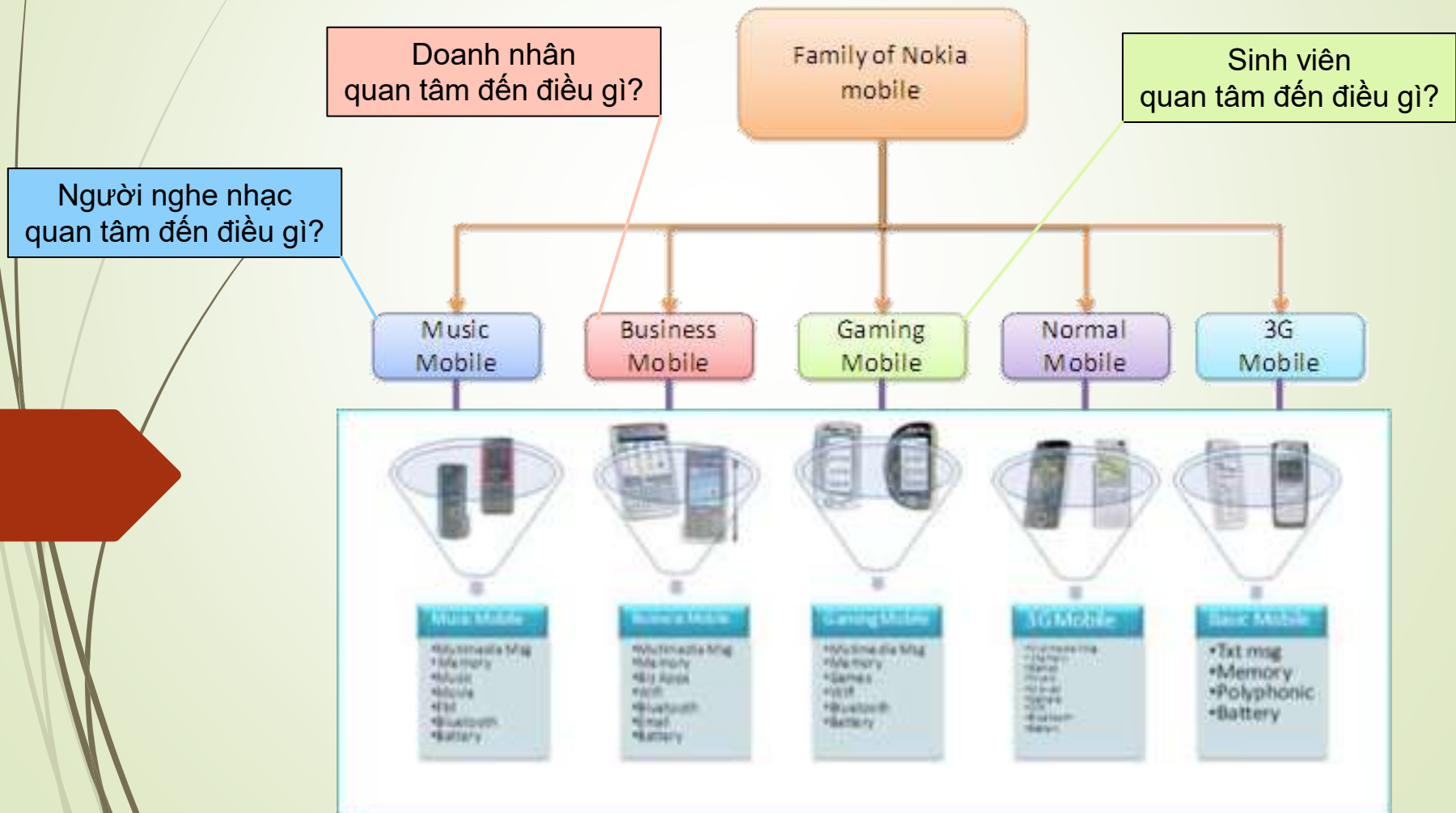
- Tổ chức là đối tượng điện thoại này vào các đặc tính chung



- Vẫn còn rất khái quát

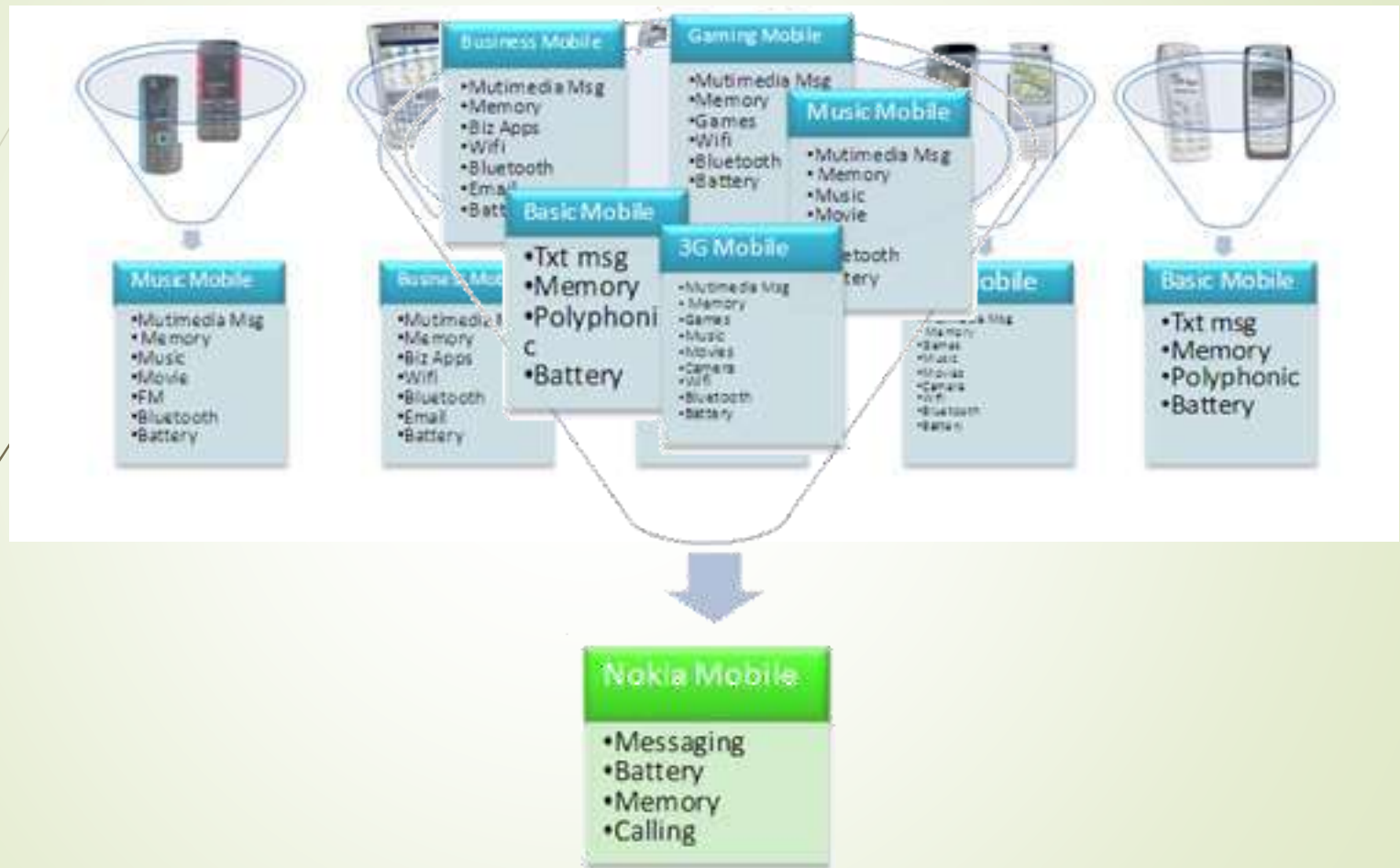
Ví dụ: Điện thoại Nokia

- Chia thành các danh mục nhỏ hơn
 - Ví dụ: Theo chức năng
- Xác định các đặc tính riêng



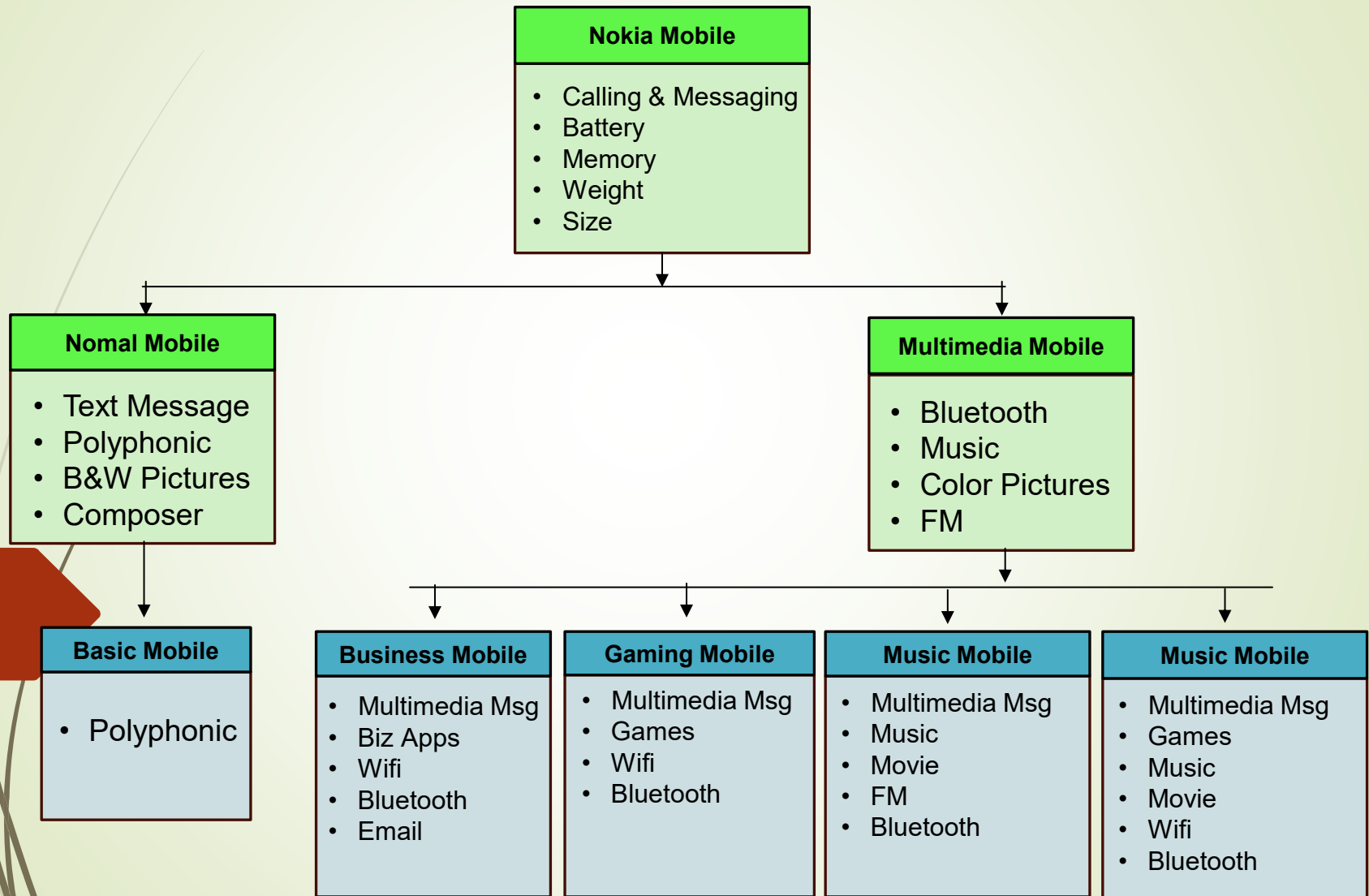
Ví dụ: Điện thoại Nokia

- Quay lại khái quát



Ví dụ: Điện thoại Nokia

- Trừu tượng hóa các đối tượng điện thoại Nokia



2



Lớp và thành phần của lớp

Các khái niệm chung về lớp và các thành phần của lớp

Lớp

- **Lớp** là cách phân loại (classify) đối tượng dựa trên đặc điểm chung.
- Lớp là **khuôn mẫu** để tạo ra các đối tượng. Ví dụ: *Người, Sinh Vật, Màu sắc...*
- Lớp là **kết quả** của quá trình trừu tượng hóa dữ liệu — định nghĩa một **kiểu dữ liệu mới**.
- **Đối tượng** (Object) là một *thể hiện* (instance) của lớp.



Lớp = bản thiết kế (blueprint)



Đối tượng = được xây từ bản thiết kế đó

Các thành phần của lớp

- Lớp đóng gói các phương thức và thuộc tính chung của các đối tượng cùng một loại



Thuộc tính (Attribute)

- Trạng thái chung được đặt tên
- Khai báo bên trong lớp như biến
- Mỗi đối tượng có bản sao riêng
- Ví dụ Ô tô: màu sắc, vận tốc



Phương thức (Method)

- Xác định hoạt động của đối tượng
- Khai báo như hàm, bên trong lớp
- Thường thao tác trên thuộc tính
- Ví dụ Ô tô: tăng tốc, giảm tốc

Lớp (Class Template)

Book

```
- authorName : String
- pageCount : int
- title : String

+ borrow()
+ getAuthorName() : String
+ countPages() : int
```

Thể hiện (Instance)

book1 : Book

```
authorName = "Vu Thi Huong Giang"
pageCount = 250
title = "Human Computer
Interaction"
```

-

private

Chỉ truy cập trong lớp

+

public

Truy cập từ mọi nơi

Phạm vi truy cập

Access Modifier

private (-)

Phạm vi hẹp nhất. Chỉ truy cập được bên trong lớp đó. Khai báo với tiền tố `__` trong Python.

default (mặc định)

Truy cập trong cùng package. Không có từ khóa đặc biệt.

public (+)

Phạm vi rộng nhất. Truy cập được từ bất cứ đâu, kể cả ngoài package.

Vị trí	+ public	mặc định	- private
Cùng lớp	✓	✓	✓
Cùng gói	✓	✓	✗
Khác gói	✓	✗	✗

3

Xây dựng lớp

Xây dựng các lớp cùng các thuộc tính và phương thức của nó trong Python

Module trong Python

- Trong Python, **modules** là những file có phần mở rộng **.py**, các file này chứa mã Python và đó có thể là các biến, hàm, hoặc một lớp nào đó.
- Các module giúp chúng ta code một cách linh hoạt hơn, mỗi file sẽ chứa những đoạn code phục vụ cho một chức năng cụ thể, được sắp xếp tùy vào cách thiết kế của mỗi lập trình viên.
- Để sử dụng các hàm của module A trong module B thì ta phải sử dụng từ khóa **import**, sau đó là bạn có thể sử dụng toàn bộ tài nguyên của module A.

Module trong Python

- `print (help('modules'))`

```
1  _future_      threading_local  getpass          sched
2  _abc          tkinter         gettext          secrets
3  _aix_support  tracemalloc    glob             select
4  _ast          uuid           graphlib         selectors
5  _asyncio      warnings       gzip            setuptools
6  _bisect       weakref        hashlib         shelve
7  _blake2       weakrefset     heapq           shlex
8  _bootlocale   winapi         helloworld      shutil
9  _bootstrap    xxsubinterpreters hmac             signal
10 _bz2          zoneinfo       html            site
11 _codecs       abc            http            smtpd
12 _codecs_cn    aifc           idlelib         smtplib
13 _codecs_hk    antigravity    imaplib         sndhdr
14 _codecs_iso2022 argparse       imgchr         socket
15 _codecs_jp    array          imp             socketserver
16 _codecs_kr    ast            importlib       sqlite3
17 _codecs_tw    asyncio        inspect        sre_compile
18 _collections  asyncio        io              sre_constants
19 _collections_abc asyncore       ipaddress      sre_parse
20 _compat_pickle atexit         itertools      ssl
21 _compression  audioop        json            stat
22 _contextvars  base64         keyword         statistics
23 _csv          bdb            lib2to3         string
24 _ctypes       binascii       linecache       stringprep
25 _ctypes_test  binhex        locale          struct
26 _datetime     bisect         logging         students
27 _decimal      builtins       lzma            subprocess
28 _distutils_hack bz2            mailbox         sunau
29 _elementtree  cProfile       mailcap         symbol
30 _functools    calendar      marshal         symtable
31 _hashlib      cgi           math            sys
32 _heapq        cgitb         minetypes       sysconfig
33 _imp          chunk          mmap            tabnanny
34 _io          cmath         modulefinder    tarfile
35 _json         cmd           msilib          telnetlib
36 _locale      code          multiprocessing tempfile
37 _lsprof       codecs        netrc           test
38 _lzma         codeop        nntplib         textwrap
39 _markupbase   collections   nt              this
40 _md5          colorsys      nt              threading
```

```
41 _msi          compileall     ntpath           time
42 _multibytecodec concurrent     nturl2path       timeit
43 _multiprocessing configparser  numbers          tkinter
44 _opcode       contextlib     opcode           token
45 _operator     contextvars   operator         tokenize
46 _osx_support  copy          optparse         trace
47 _overlapped   copyreg       os              traceback
48 _peg_parser   crypt         parser           tracemalloc
49 _pickle       csv           pathlib         tty
50 _py_abc       ctypes        pdb             turtle
51 _pydecimal    curses        pickle           turtledemo
52 _pyio         dataclasses  pickletools     types
53 _queue        datetime     pip             typing
54 _random       dbm           pipes           unicodedata
55 _sha1         decimal      pkg_resources   unittest
56 _sha256       difflib      pkgutil         urllib
57 _sha3         dis          platform        uu
58 _sha512       distutils    plistlib        uuid
59 _signal       doctest      poplib          venv
60 _sitebuiltins easy_install  posixpath       warnings
61 _socket       email        pprint          wave
62 _sqlite3      encodings    profile         weakref
63 _sre          ensurepip    pstats         webbrowser
64 _ssl          enum         pty            winreg
65 _stat         errno        py_compile     winsound
66 _statistics   faulthandler pycbr          wagiref
67 _string       filecmp      pydoc           xdrlib
68 _strptime     fileinput    pydoc_data      xml
69 _struct       fnmatch     pyexpat         xmlrpc
70 _symtable     formatter    queue           xxsubtype
71 _testbuffer   fractions    quopri          zipapp
72 _testcapi     ftplib      random          zipfile
73 _testconsole  functools    re              zipimport
74 _testimportmultiple gc            reprlib         zlib
75 _testmultiphase genericpath   rcompleter      zoneinfo
76 _thread       getopt      runpy
```

BTVN: Sinh viên tự tìm hiểu cách tạo, gọi module trong python

Gói

- Gói (package) giống như thư mục giúp:
 - Tổ chức và xác định vị trí lớp dễ dàng và sử dụng các lớp một cách phù hợp
 - Tránh cho việc đặt tên lớp bị xung đột (trùng tên)
 - Các package khác nhau có thể chứa các lớp có cùng tên
 - Bảo vệ các lớp, dữ liệu và phương thức ở mức rộng hơn so với mối quan hệ giữa các lớp.
- Một package cũng có thể chứa các package khác
- Còn được gọi là không gian tên (namespace) trong một số ngôn ngữ lập trình (C/C++...)

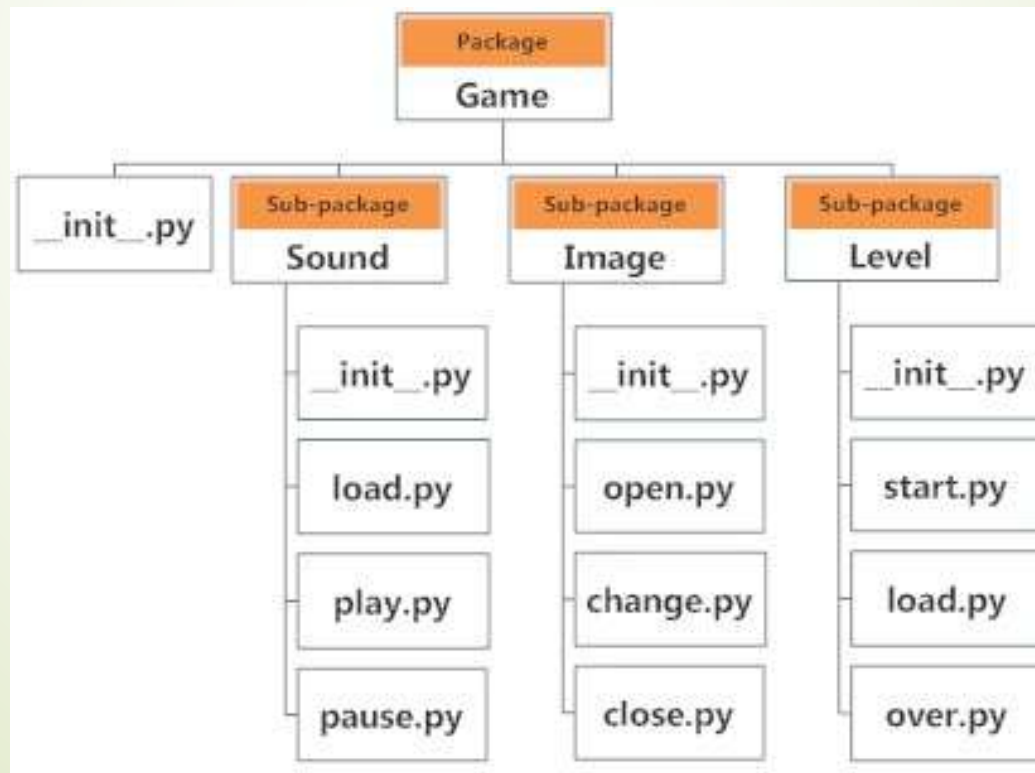
Gói trong Python

- in a Python shell/prompt: `pip list`

Package	Version		
absl-py	1.4.0	click	8.0.3
alabaster	0.7.12	cloudpickle	2.0.0
anaconda-client	1.9.0	clyent	1.2.2
anaconda-navigator	2.1.1	colorama	0.4.4
anaconda-project	0.10.1	conda	4.10.3
anyio	2.2.0	conda-build	3.21.5
appdirs	1.4.4	conda-content-trust	0+unknown
applaunchservices	0.2.1	conda-pack	0.6.0
appnope	0.1.2	conda-package-handling	1.7.3
appsript	1.1.2	conda-repo-cli	1.0.4
argh	0.26.2	conda-token	0.3.0
argon2-cffi	20.1.0	conda-verify	3.4.2
arrow	0.13.1	contextlib2	0.6.0.post1
asgiref	3.5.2	contourpy	1.0.5
asn1crypto	1.4.0	cookiecutter	1.7.2
astroid	2.6.6	cryptography	3.4.8
astropy	4.3.1	cycler	0.10.0
astunparse	1.6.3	Cython	0.29.24
async-generator	1.10	cytoolz	0.11.0
atomicwrites	1.4.0	daal4py	2021.3.0
attrs	21.2.0	dask	2021.10.0
autopep8	1.5.7	debugpy	1.4.1
Babel	2.9.1	decorator	5.1.0
backcall	0.2.0	defusedxml	0.7.1
backports.functools-lru-cache	1.6.4	diff-match-patch	20200713
backports.shutil-get-terminal-size	1.0.0	distlib	0.3.6
backports.tempfile	1.0	distributed	2021.10.0
backports.weakref	1.0.post1	Django	4.1.2
beautifulsoup4	4.10.0	docutils	0.17.1
binaryornot	0.4.4	entrypoints	0.3
bitarray	2.3.0	et-xmlfile	1.1.0
bkcharts	0.2	fastcache	1.1.0
black	21.12b0	filelock	3.8.0
bleach	4.0.0	flake8	3.9.2
bokeh	2.4.1	Flask	1.1.2
boto	2.49.0	flatbuffers	23.3.3
Bottleneck	1.3.2	fonttools	4.25.0
brotlipy	0.7.0	fsspec	2021.8.1
cached-property	1.5.2	future	0.18.2
cachetools	5.3.0	gast	0.4.0
certifi	2022.12.7	gevent	21.8.0
cffi	1.14.6	glob2	0.7
cfgv	3.3.1	gmpy2	2.0.8
chardet	4.0.0	google-auth	2.16.2
charset-normalizer	3.1.0	google-auth-oauthlib	0.4.6
		google-pasta	0.2.0
		greenlet	1.1.1
		grpcio	1.51.3

Ví dụ: Gói

- Hình ảnh dưới đây là cấu trúc của 1 package **Game**. Bên trong Game có thêm 3 package con là **Sound**, **Image**, **Level**
- Bên trong 3 package con này chứa các module có liên quan với nhau.
- Điều đặc biệt là mỗi package **bắt buộc** phải có file `__init__.py`, đây là file được gọi đầu tiên mỗi khi bạn import package nó vào chương trình. Nếu bạn đã học lập trình hướng đối tượng thì file này giống như **hàm khởi tạo** của một lớp 1.



Khai báo lớp

- Chúng ta đã sử dụng nhiều kiểu tích hợp sẵn (build-in types) của Python; bây giờ chúng ta sẽ xác định một kiểu mới. Ví dụ, chúng ta sẽ tạo một kiểu gọi là **Point** đại diện cho một điểm trong không gian hai chiều. Kiểu **Point** hay lớp **Point** còn gọi là Kiểu tự định nghĩa (Programmer-defined types)



Khai báo lớp

- Trong ký hiệu toán học, các điểm thường được viết trong ngoặc đơn với dấu phẩy ngăn cách các tọa độ. Ví dụ: $(0, 0)$ đại diện cho điểm gốc, và (x, y) đại diện cho tọa độ x trên trục hoành và y trên trục tung. Có một số cách chúng ta có thể biểu diễn các điểm trong Python:
 - Chúng ta có thể lưu trữ các tọa độ riêng biệt trong hai **biến**, x và y .
 - Chúng ta có thể lưu trữ các tọa độ dưới dạng các phần tử trong một **list** hoặc bộ dữ liệu **tuples**.
 - Chúng ta có thể tạo một **kiểu mới** để biểu diễn các điểm dưới dạng các đối tượng.

Khai báo lớp

- Tạo một kiểu mới có vẻ phức tạp hơn các tùy chọn khác, nhưng nó có những ưu điểm sẽ sớm rõ ràng.
- Kiểu do người lập trình xác định còn được gọi là lớp (**class**). Một định nghĩa lớp trông như thế này:

```
class Point:
```

```
    " " " Represents a point in 2-D space. " " "
```

- Tiêu đề chỉ ra rằng lớp mới được gọi là **Point**. Phần thân là một chuỗi tài liệu (docstrings) giải thích những gì mà lớp dành cho. Bạn có thể xác định các thuộc tính (attributes) và phương thức (methods) bên trong định nghĩa lớp.

Ví dụ: Khai báo lớp

```
point.py

class Point:
    """Represents a point in 2-D space."""
    x = int
    y = int

    def print_point(self):
        print('( %d, %d)' % (
            self.x, self.y))

# Tạo đối tượng
p1 = Point()
p1.x = 3
p1.y = 5
p1.print_point()
# Kết quả: (3, 5)
```



`class Point:`

Từ khóa class khai báo lớp. Tên lớp theo PascalCase.



`"""..."""`

Docstring mô tả lớp — Programmer-defined type.



`x = int, y = int`

Khai báo thuộc tính lớp. Mỗi đối tượng có bản sao riêng.



`def print_point(self):`

self là tự tham chiếu đến đối tượng hiện tại.



`p1 = Point()`

Khởi tạo đối tượng. `p1.x = 3` — gán giá trị thuộc tính.

Thuộc tính

- Là các thông tin, trạng thái mà đối tượng của lớp đó có thể mang
- Các thuộc tính phải được khai báo bên trong lớp
- Mỗi đối tượng có bản sao các thuộc tính của riêng nó
 - Giá trị của một thuộc tính thuộc các đối tượng khác nhau là khác nhau.
- Bản chất của các thuộc tính là các thành phần dữ liệu của đối tượng
 - Khai báo: tương tự như biến

Phương thức

- Xác định cách một đối tượng đáp ứng lại thông điệp
 - Khai báo: tương tự khai báo hàm
- Phương thức xác định các hoạt động của lớp
- Bất kỳ phương thức nào cũng phải thuộc về một lớp nào đó



Ví dụ: Thuộc tính & Phương thức

Khai báo lớp

Khai báo thuộc tính

Khai báo hàm
(nằm ngoài class)

Khởi tạo đối tượng

Gán giá trị cho thuộc tính

```
class Point:
    x = int
    y = int

def print_point(point):
    print ('(%d, %d)' % (point.x, point.y))

p1 = Point()
p1.x = 3
p1.y = 5
print_point(p1)
```

Kết quả: (3, 5)

Ví dụ: Thuộc tính & Phương thức

Khai báo lớp

Khai báo thuộc tính

Khai báo phương thức

Khởi tạo đối tượng

Gán giá trị cho thuộc tính

Gọi phương thức

```
class Point:
    x = int
    y = int

    def print_point(self):
        print ('(%d, %d)' % (self.x,self.y))

p1 = Point()
p1.x = 3
p1.y = 5
p1.print_point()
```

Kết quả: (3, 5)

Ví dụ: Thuộc tính & Phương thức

- So sánh sự khác nhau của hai đoạn mã lệnh trên

```
class Point:
    x = int
    y = int

def print_point(point):
    print ('(%d, %d)' % (point.x, point.y))
```

```
p1 = Point()
p1.x = 3
p1.y = 5
print_point(p1)
```

Kết quả: (3, 5)

Thụt lề

```
class Point:
    x = int
    y = int

def print_point(self):
    print ('(%d, %d)' % (self.x, self.y))
```

```
p1 = Point()
p1.x = 3
p1.y = 5
p1.print_point()
```

Kết quả: (3, 5)

Tự tham chiếu

Tự tham chiếu đến
phương thức

Tự tham chiếu

- Sử dụng từ khóa **self**
- Cho phép truy cập vào đối tượng hiện tại của lớp.
- Quan trọng khi hàm/phương thức thành phần thao tác trên hai hay nhiều đối tượng.
- Xóa đi sự nhập nhằng giữa một biến cục bộ, tham số với thành phần dữ liệu của lớp

Chỉ định truy cập

- Chỉ định truy cập:
 - public**: Lớp có thể được truy cập từ bất cứ đâu, kể cả bên ngoài package chứa lớp đó.
 - private**: Lớp chỉ có thể được truy cập trong phạm vi lớp đó.
 - mặc định: Lớp có thể được truy cập từ bên trong package chứa lớp đó.

inner class

	public	mặc định	private
Cùng lớp	✓	✓	✓
Cùng gói	✓	✓	✗
Khác gói	✓	✗	✗

```
class Point:  
    x = int  
    y = int
```

```
class Point:  
    _x = int  
    _y = int
```

access modifier

Phương thức Get & Set — Getter / Setter

Getter – Phương thức truy vấn

Hỏi giá trị thuộc tính của đối tượng. Không thay đổi trạng thái.

3 kiểu truy vấn:

- Đơn giản: "giá trị x là bao nhiêu?"
- Điều kiện: "x có > 10 không?"
- Dẫn xuất: "tổng x + y là bao nhiêu?"

Setter – Phương thức thiết lập

Thay đổi giá trị thuộc tính của đối tượng.

Ưu điểm quan trọng:

- Kiểm tra tính hợp lệ trước khi gán
- Bảo vệ dữ liệu khỏi giá trị không hợp lệ
- Đảm bảo tính toàn vẹn (data integrity)

```
class Point:
    __x = int  # private: __ prefix
    __y = int

    def getX(self):      # Getter – chỉ đọc, không sửa
        return self.__x

    def getY(self):
        return self.__y

    def setXY(self, x=3, y=5):  # Setter – có thể kiểm tra hợp lệ
        if isinstance(x, int) and isinstance(y, int):
            self.__x = x
            self.__y = y

p1 = Point()
print(p1.getX())  # <class 'int'>
p1.setXY(3, 5)
print(p1.getX())  # 3
```

4



Tạo và sử dụng đối tượng

Khai báo và khởi tạo đối tượng. Truy cập đến các phương thức và thuộc tính của đối tượng

Ví dụ thực hành

1

Lớp Point — 2D Geometry

Lớp **Point** với thuộc tính $x, y : \text{int}$. Yêu cầu:

- Tạo điểm $A(3, 4) \rightarrow$ hiển thị tọa độ
- Tạo điểm B từ bàn phím
- Tạo điểm C đối xứng với B qua gốc O
- Tính khoảng cách từ B đến O: $d = \sqrt{x^2 + y^2}$
- Tính khoảng cách từ A đến B: $d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

2

Lớp siêu nhân

Tạo lớp **SieuNhan** với thuộc tính: `ten`, `vu_khi`, `mau_sac`.

Khởi tạo 2 đối tượng:

- `sieu_nhan_A`: tên A, kiếm, màu đỏ
- `sieu_nhan_B`: tên B, khiên, màu xanh

In ra thể hiện của cả hai để kiểm tra.

3

Quản lý danh sách siêu nhân

Chương trình quản lý siêu nhân toàn thế giới:

- Khởi tạo class siêu nhân (thuộc tính tùy chọn)
- Dùng **while** để nhập danh sách
- Dùng **for** để in danh sách kèm thuộc tính

5



Biến lớp, biến tĩnh

Class and Static variable

Phương thức Set

- Trong lập trình hướng đối tượng
 - Các thành viên bình thường là thành viên thuộc về đối tượng
 - Các thành viên tĩnh (static) hoặc có thể gọi là thành viên lớp (class) là các thành viên thuộc về lớp
- Khi chúng ta khai báo một biến bên trong một lớp nhưng bên ngoài bất kỳ phương thức nào, nó được gọi là biến lớp hoặc biến tĩnh trong Python. Biến lớp hoặc biến tĩnh có thể được giới thiệu thông qua một lớp nhưng không trực tiếp thông qua một thể hiện.
- Biến lớp hoặc biến tĩnh khá khác biệt và không xung đột với bất kỳ biến thành viên nào khác có cùng tên. Dưới đây là chương trình minh họa việc sử dụng lớp hoặc biến tĩnh

Ví dụ: biến tĩnh, biến thể hiện

```
# Python program to show that the variables with a value  
# assigned in class declaration, are class variables
```

```
# Class for Computer Science Student
```

```
class CSStudent:
```

```
    stream = 'cse' # Class Variable
```

```
    def __init__(self, name, roll):
```

```
        self.name = name # Instance Variable
```

```
        self.roll = roll # Instance Variable
```

```
# Objects of CSStudent class
```

```
a = CSStudent('Geek', 1)
```

```
b = CSStudent('Nerd', 2)
```

```
print(a.stream) # prints "cse"
```

```
print(b.stream) # prints "cse"
```

```
print(a.name) # prints "Geek"
```

```
print(b.name) # prints "Nerd"
```

```
print(a.roll) # prints "1"
```

```
print(b.roll) # prints "2"
```

```
# Class variables can be accessed using class
```

```
# name also
```

```
print(CSStudent.stream) # prints "cse"
```

```
# Now if we change the stream for just a it won't be changed for b
```

```
a.stream = 'ece'
```

```
print(a.stream) # prints 'ece'
```

```
print(b.stream) # prints 'cse'
```

```
# To change the stream for all instances of the class we can change it
```

```
# directly from the class
```

```
CSStudent.stream = 'mech'
```

```
print(a.stream) # prints 'ece'
```

```
print(b.stream) # prints 'mech'
```

biến tĩnh

biến thể hiện

Output:

cse

cse

Geek

Nerd

1

2

cse

ece

cse

ece

mech

Ví dụ: biến tĩnh, biến thể hiện

```
class MyClass:
    static_var = 0   biến tĩnh

    def __init__(self):
        MyClass.static_var += 1
        self.instance_var = MyClass.static_var

obj1 = MyClass()
print(obj1.instance_var) # Output: 1

obj2 = MyClass()
print(obj2.instance_var) # Output: 2

print(MyClass.static_var) # Output: 2
```

 biến thể hiện

6



Biểu đồ lớp

Biểu diễn lớp trong UML

Biểu đồ lớp (class diagram)

CẤU TRÚC BIỂU ĐỒ LỚP UML

Tên lớp

① Tên lớp

Hàng đầu tiên, in đậm, căn giữa. Tên theo PascalCase.

- +

② Cấu trúc (Attributes)

Danh sách thuộc tính với visibility ký hiệu. Dấu gạch chân = static.

f()

③ Hành vi (Methods)

Danh sách phương thức với visibility, tham số và kiểu trả về.

Ký hiệu visibility:

- + public – truy cập mọi nơi
- private – chỉ trong lớp
- # protected – lớp + lớp con
- underline static – thuộc về lớp

Professor

```
- name
- employeeID : UniqueId
- hireDate
- status
- discipline
- maxLoad

+ submitFinalGrade ()
+ acceptCourseOffering ()
+ setMaxLoad ()
+ takeSabbatical ()
+ teachClass ()
```

Student

```
- name
- address
- studentID
- nextAvailID : int

+ addSchedule (...)
+ hasPrerequisites (...) : boolean
+ getNextAvailID () : int
```

Chỉ định truy cập

- Trong biểu đồ lớp, dấu **-** được sử dụng để thể hiện chỉ định truy cập private
- Dấu **+** được sử dụng để thể hiện chỉ định truy cập public

Professor

- name
- employeeID : UniqueId
- hireDate
- status
- discipline
- maxLoad

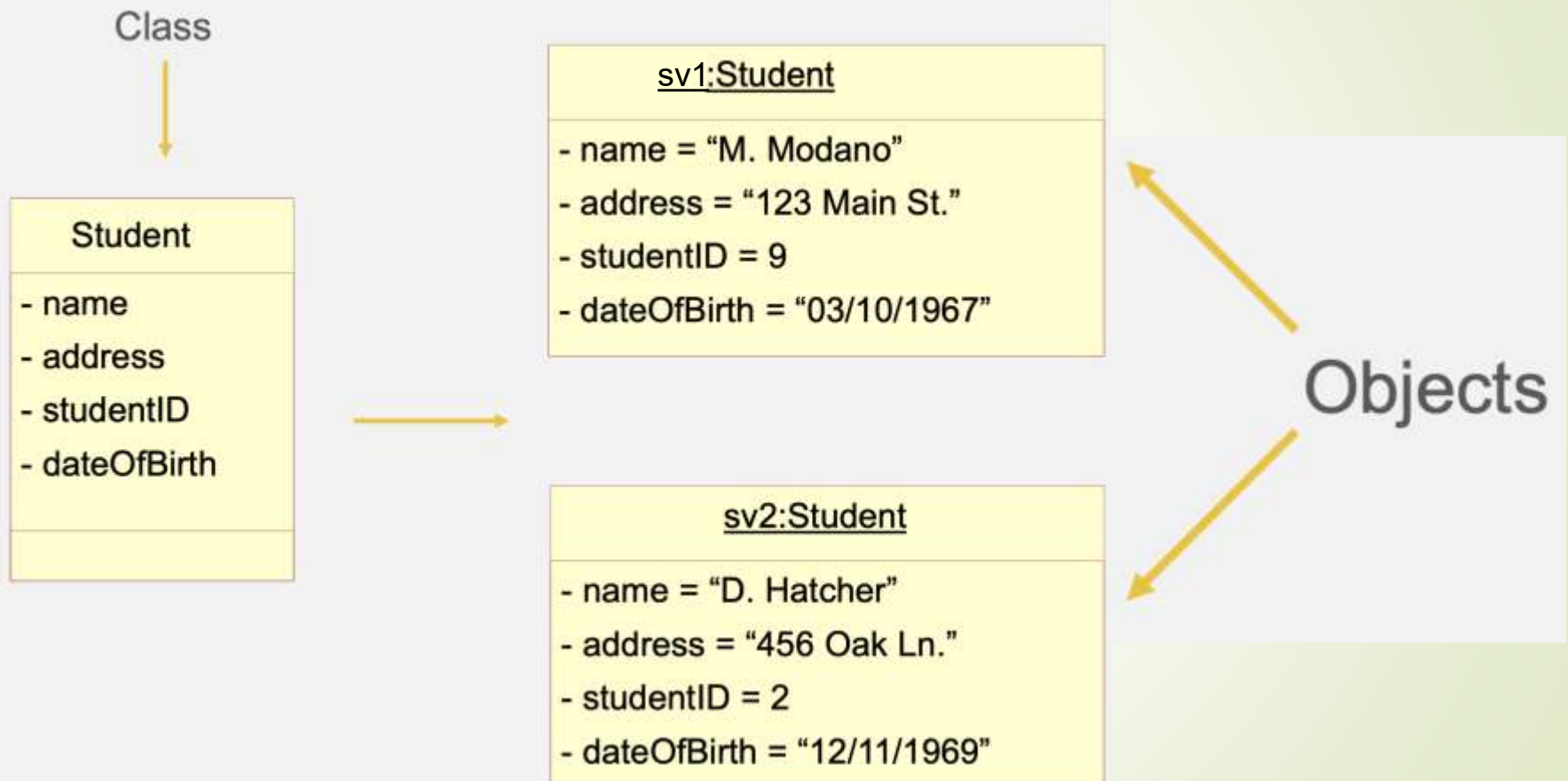
+ submitFinalGrade()
+ acceptCourseOffering()
+ setMaxLoad()
+ takeSabbatical()
+ teachClass()

Giá trị tĩnh

- Giá trị tĩnh biểu diễn bằng dấu gạch chân



Lớp và đối tượng trong UML



Ví dụ

- Lớp BankAccount

BankAccount
- owner: String - balance: double
+ debit(double): boolean + credit(double)

- Lớp Student

Student
- name - address - studentID - dateOfBirth

Ask questions!



Bài tập

Nhận diện lớp từ thực tế

Đối tượng	Thuộc tính	Phương thức
 Con chó	Tên, Màu sắc, Giống, Cảm xúc	Sữa, Vẫy đuôi, Ăn, Chạy
 Ô tô	Hãng, Kích thước, Màu, Giá	Tăng tốc, Giảm tốc, Đâm
 Tài khoản	Tên TK, Số TK, Ngân hàng, Số dư	Rút, Gửi, Kiểm tra số dư

Bài tập

#Think Python, Chapter 15

Exercise 15.1. Write a definition for a class named `Circle` with attributes `center` and `radius`, where `center` is a `Point` object and `radius` is a number.

Instantiate a `Circle` object that represents a circle with its center at (150, 100) and radius 75.

Write a function named `point_in_circle` that takes a `Circle` and a `Point` and returns `True` if the `Point` lies in or on the boundary of the circle.

Write a function named `rect_in_circle` that takes a `Circle` and a `Rectangle` and returns `True` if the `Rectangle` lies entirely in or on the boundary of the circle.

Write a function named `rect_circle_overlap` that takes a `Circle` and a `Rectangle` and returns `True` if any of the corners of the `Rectangle` fall inside the `Circle`. Or as a more challenging version, return `True` if any part of the `Rectangle` falls inside the `Circle`.

Circle

+ `center : Point`

+ `radius : float`

+ `point_in_circle()`

+ `rect_in_circle()`

+ `rect_circle_overlap()`

Bài tập

BÀI TẬP NÂNG CAO: LỚP NHANVIEN

NhanVien

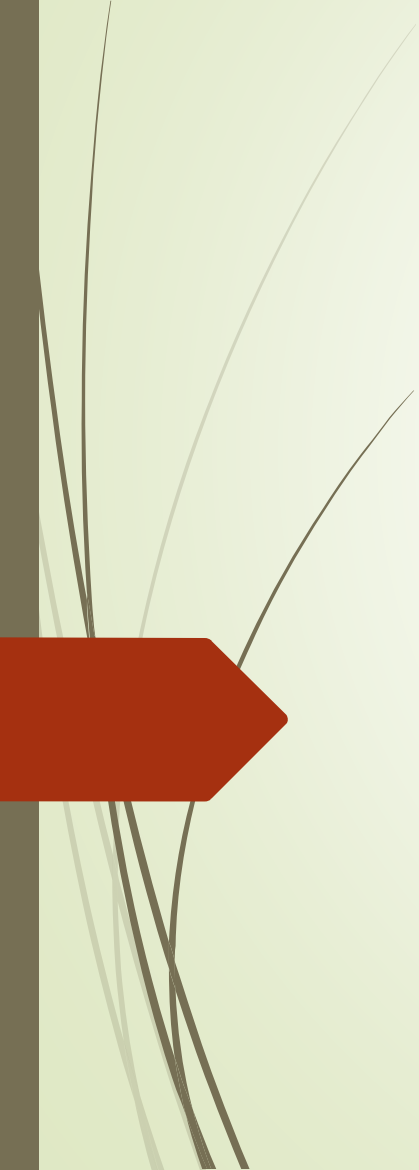
- tenNhanVien : String
- luongCoBan : double
- heSoLuong : double
- + LUONG_MAX : double
- + tangLuong(double) : boolean
- + tinhLuong() : double
- + inTTin()

1 **tinhLuong()**: Lương = $\text{luongCoBan} \times \text{heSoLuong}$

2 **inTTin()**: Hiển thị đầy đủ thông tin nhân viên.

3 **tangLuong(delta)**: Nếu lương mới > LUONG_MAX → in thông báo, trả về False. Ngược lại → trả về True.

4 Viết đầy đủ **getter/setter** cho tất cả thuộc tính.

A decorative graphic on the left side of the slide featuring several thin, curved lines representing grass or reeds. At the bottom left, there is a solid red arrow pointing to the right.

Thank you!

Any questions?